

Extending E Prover with Similarity Based Clause Selection Strategies

Jan Jakubův^(✉) and Josef Urban

CIIRC, Czech Technical University, Prague, Czech Republic
jakubuv@gmail.com, josef.urban@gmail.com

Abstract. E prover is a state-of-the-art theorem prover for first-order logic with equality. E prover is built around a saturation loop, where new clauses are derived by inference rules from previously derived clauses. Selection of clauses for the inference provides the main source of non-determinism and an important choice-point of the loop where the right choice can dramatically influence the proof search. In this work we extend E Prover with several new clause selection strategies based on similarity of a clause with the conjecture. In particular, clauses which are more related to the conjecture are preferred. We implement different strategies that define the relationship with a conjecture in different ways. We provide an implementation of the proposed selection strategies and we evaluate their efficiency on an extensive benchmark set.

Keywords: Automated theorem proving · Large theory reasoning · Clause selection

1 Introduction

Many state-of-the-art automated theorem provers (ATPs) are based on the *given clause algorithm* introduced by Otter [5]. The input problem $T \cup \{\neg C\}$ is translated into a refutationally equivalent set of clauses. Then the search for a contradiction, represented by the empty clause, is performed maintaining two sets: the set P of *processed clauses* and the set U of *unprocessed clauses*. Initially, all the input clauses are unprocessed. The algorithm repeatedly selects a *given clause* g from U and generates all possible inferences using g and the processed clauses from P . Then, g is moved to P , and U is extended with the newly produced clauses. This process continues until a resource limit is reached, or the empty clause is inferred, or P becomes *saturated*, that is, nothing new can be inferred.

The search space of this loop grows quickly. Several methods can be used to make the proof search more efficient. The search space can be narrowed by adjusting (typically restricting) the inference rules, pruned by using *forward* and *backward subsumption*, reduced by pre-selecting relevant input clauses, or otherwise simplified. One of the main sources of non-determinism affecting efficiency

J. Jakubův and J. Urban—Supported by the ERC Consolidator grant nr. 649043 *AI4REASON*.

of the search is the selection of the given clause. Clever selection mechanism can improve the search dramatically: in principle, one only needs to do the inferences that participate in the final proof. So far, this is often only a tiny portion of all the inferences done by the ATPs during the proof search.

2 Clause Selection in E Prover

E [6] is a state-of-the-art theorem prover which we use as a basis for implementation. The selection of a given clause in E is implemented by a combination of priority and weight functions. A *priority function* assigns an integer to a clause and is used to pre-order clauses for weight evaluation. A *weight function* takes additional specific arguments and assigns to each clause a real number called *weight*. A *clause evaluation function* *CEF* is specified by a priority function, weight function, and its arguments. Each *CEF* selects the clause with the smallest pair (*priority*, *weight*) for inferences. E allows a user to select an *expert heuristic* on a command line in the format “($n_1 * CEF_1, \dots, n_k * CEF_k$)”, where integer n_i indicates how often the corresponding CEF_i should be used to select a given clause. E additionally supports an *autoschedule* mode where several expert heuristics are tried, each for a selected time period. The heuristics and time periods are automatically chosen based on input problem properties.

One of the well-performing weight functions in E, which we also use as a reference for evaluation of our weight functions, is the *conjecture symbol weight*. This weight function counts symbol occurrences with different weights based on their appearance in the conjecture as follows. Different weights δ_f , δ_c , δ_p , and δ_v are assigned to function, constant, and predicate symbols, and to variables. The weight of a symbol which appears in the conjecture is multiplied by γ_{conj} , typically $\gamma_{\text{conj}} < 1$ to prefer clauses with conjecture symbols. To compute a term weight, the given symbol weights are summed for all symbol occurrences. This evaluation is extended to equations and to clauses.

3 Similarity Based Clause Selection Strategies

Many of the best-performing weight functions in E are based on a similarity of a clause with the conjecture, for example, the *conjecture symbol weight* from the previous section. In this paper we try to answer the question whether or not it makes sense to also investigate a term structure. We propose, implement, and evaluate several weight functions which utilize conjecture similarity in different ways. Typically they extend the symbol-based similarity by similarity on terms. Using finer formula features improves the high-level premise selection task [2], which motivates this work on steering also the internal selection in E. We first describe the common arguments of our weight functions and then function-specific properties.

Common Arguments (v, r, e). We implement two ways of term variable normalization, selected by the argument v . Either (1) variables are α -normalized, naming them consistently by their appearance in the term from left to right

(value “ α ”), or (2) all variables are unified to a single variable (“ $\star$ ”). This provides differently coarse notions of similarity. Each of our weight functions relates a term to the global set `RelatedTerms`. This set `RelatedTerms`, controlled by the argument `r`, contains either (1) all conjecture terms (“`ter`”), (2) conjecture terms and their subterms (“`sub`”), (3) conjecture subterms and top-level generalizations (“`top`”), or to (4) conjecture subterms and all their generalizations (“`gen`”). Each of our weight functions implements a different function `base-weight` which assigns a weight to a term. We use three different ways of extending `base-weight` to compute a term weight, selected by the argument `e`. Either (1) `base-weight` value is used directly (value “1”), or (2) values of `base-weight` for all the subterms are summed (“ Σ ”), or (3) the maximal value of `base-weight` on all of the subterms is used (“ $\vee$ ”).

Conjecture Subterm Weight (Term). The first of our weight functions is similar to the standard *conjecture symbol weight*, counting instead of symbols the number of subterms a term shares with the conjecture. The weight function `Term` takes five specific arguments γ_{conj} , δ_f , δ_c , δ_p and δ_v and `base-weightTerm(t)` equals weight δ_f for functional terms, δ_c for constants, δ_p for predicates, and δ_v for variables, possibly multiplied by γ_{conj} when $t \in \text{RelatedTerms}$.

Conjecture Frequency Weight (Tfidf). *Term frequency – inverse document frequency*, is a numerical statistic intended to reflect how important a word is to a document in a corpus [3]. A *term frequency* is the number of occurrences of the term in a given document. A *document frequency* is the number of documents in a corpus which contain the term. The term frequency is typically multiplied by the logarithm of the inverse of document frequency to reduce frequency of terms which appear often. We define $\text{tf}(t)$ as the number of occurrences of t in `RelatedTerms`. We consider a fixed set of clauses denoted `Docs`. We define $\text{df}(t)$ as the count of clauses from `Docs` which contain t . Our weight function `Tfidf` takes one specific argument δ_{doc} to select documents, either (1) `ax` for the axioms or (2) `pro` for all the processed clauses, and `base-weightTfidf` is as follows.

$$\text{base-weight}_{\text{Tfidf}}(t) = \frac{1}{1 + \text{tfidf}(t)} \quad \text{where} \quad \text{tfidf}(t) = \text{tf}(t) * \log \frac{1 + |\text{Docs}|}{1 + \text{df}(t)}$$

Conjecture Term Prefix Weight (Pref). The above weight functions rely on an exact match of a term with a conjecture related term. The following weight function loosen this restriction and consider also partial matches. We consider terms as symbol sequences. Let `max-pref(t)` be the longest prefix t shares with a term from `RelatedTerms`. A *term prefix weight* (`Pref`) counts the length of `max-pref(t)` using weight arguments δ_{match} and δ_{miss} , formally, `base-weightPref(t)` = $\delta_{\text{match}} * |\text{max-pref}(t)| + \delta_{\text{miss}} * (|t| - |\text{max-pref}(t)|)$.

Conjecture Levenshtein Distance Weight (Lev). A straightforward extension of `Pref` is to employ the Levenshtein distance [4] which measures a distance of two strings as the minimum number of edit operations (character insertion, deletion, or change) required to change one word into the other. Our weight function `Lev` defines `base-weightLev(t)` as the minimal distance from t to some $s \in \text{RelatedTerms}$. It takes additional arguments δ_{ins} , δ_{del} , δ_{ch} to assign different costs for edit operations.

Conjecture Tree Distance Weight (Ted). The Levenshtein distance does not respect a tree structure of terms. To achieve that, we implement the *Tree edit distance* [8] which is similar to Levenshtein but uses tree editing operations (inserting a node into a tree, deleting a node while reconnecting its child nodes to the deleted position, and renaming a node label). Our weight function **Ted** takes the same arguments as **Lev** above and $\text{base-weight}_{\text{Ted}}$ is defined similarly.

Conjecture Structural Distance Weight (Struc). With **Ted**, a tree produced by the edit operations does not need to represent a valid term as the operations can change number of child nodes. To avoid this we define a simple *structural distance* which measures a distance of two terms by a number of *generalization* and *instantiation* operations. Generalization transforms an arbitrary term to a variable while instantiation does the reverse. Our weight function **Struc** takes additional arguments δ_{miss} , γ_{inst} , and γ_{gen} as penalties for variable mismatch and operation costs. The distance of a variable x to a term t is the cost of instantiating x to t , computed as $\Delta_{\text{Struc}}(x, t) = \gamma_{\text{inst}} * |t|$. The distance of t to x is defined similarly but with γ_{gen} . A distance of non-variable terms t and s which share the top-level symbol is the sum of distances of the corresponding arguments. Otherwise, a generic formula $\Delta_{\text{Struc}}(t, x_0) + \Delta_{\text{Struc}}(x_0, s)$ is used. Function $\text{base-weight}_{\text{Struc}}$ is as for **Lev** but using Δ_{Struc} .

4 Experimental Results and Evaluation

The best evaluation would be to measure how our weight functions enrich the autoschedule mode of E. This is, however, beyond the scope of this paper. Instead, we design experiments to help us estimate the quality of the new weights. For each new weight function we run all possible combinations of common arguments (“*v-r-e*”, see Sect. 3) and other manually selected arguments. First, we run the weight functions on the 2078 MPTP bushy problems [1] with a 5 s time limit. We compare the number of solved problems with the number of problems solved by the *conjecture symbol weight* (denoted **ref**) discussed in Sect. 2. Second, to estimate how complementary our weight functions are with existing functions, we pick a well-performing expert heuristic from the autoschedule mode of E, and we compute how many problems were solved which the expert heuristic was not able to solve in 10 s (denoted **2E+**). The five best-performing combinations of arguments for each weight function are presented in Table 1. Column *speed* contains an average number of processed (*kilo*-)clauses per second to evaluate implementation efficiency. Our implementation is available for download¹.

From Table 1 we can see that the weights which rely on an exact match of a term with a related term or its part (**Term**, **Tfldf**, and **Pref**) perform best when values of **base-weight** are summed for all the subterms ($e = \Sigma$). On the other hand, weights which incorporate some notion of term similarity directly in **base-weight** do not profit so much from this. For weights **Lev**, **Ted**, and **Struc** we have tried to experiment with operation costs (column δ , for example, 151 means that δ_{del} is increased to 5 while other costs are 1). In general, the experiments show that different arguments have an impact on performance. Finally, the experiments also reveal a higher time complexity of the **Lev** and **Ted** weights (Levenshtein distance of two terms is in $O(n^2)$ while **Ted** is in $O(n^3)$). However, a higher time complexity does not have to be a drawback as **Lev** is still best performing.

¹ <http://people.ciirc.cvut.cz/jakubja5/src/E-arg-2016-03.tar.gz>.

Table 1. The five best-performing configurations for each weight function.

Term	solved	speed	%ref+	Term	2E+	speed	%ref+
$\star$ -gen- Σ	749	5.6	5.3	α -gen-1	20	4.4	-0.7
α -gen- Σ	749	5.4	5.3	$\star$ -sub- Σ	19	5.7	1.0
$\star$ -sub- Σ	718	5.7	1.0	$\star$ -ter- Σ	19	5.7	0.8
$\star$ -ter- Σ	717	5.7	0.8	α -ter- Σ	18	5.5	0.8
α -ter- Σ	717	5.5	0.8	α -sub- Σ	18	5.5	0.6
ref	711	3.4	0.0	ref	7	3.4	0.0

Tfldf	δ_{doc}	solved	speed	%ref+	Tfldf	δ_{doc}	2E+	speed	%ref+
α -gen- Σ	pro	738	3.1	3.8	$\star$ -sub- Σ	pro	17	3.5	0.3
α -gen- Σ	ax	736	3.7	3.5	$\star$ -gen- Σ	pro	16	3.3	3.4
$\star$ -gen- Σ	pro	735	3.3	3.4	$\star$ -ter- Σ	pro	16	3.6	0.7
$\star$ -gen- Σ	ax	733	3.6	3.1	α -sub- Σ	pro	16	3.3	0.1
$\star$ -ter- Σ	pro	716	3.6	0.7	$\star$ -sub- Σ	ax	16	3.9	0.0

Pref	solved	speed	%ref+	Pref	2E+	speed	%ref+
α -gen- Σ	788	4.0	10.8	α -gen- Σ	21	4.0	10.8
α -top- Σ	772	4.2	8.6	$\star$ -gen- Σ	20	3.9	8.4
$\star$ -gen- Σ	771	3.9	8.4	α -gen-1	18	3.9	8.0
α -gen-1	768	3.9	8.0	$\star$ -sub- Σ	18	4.3	7.7
$\star$ -sub- Σ	766	4.3	7.7	α -sub- Σ	18	4.2	7.5

Lev	δ	solved	speed	%ref+	Lev	δ	2E+	speed	%ref+
$\star$ -gen-1	155	841	2.4	18.3	$\star$ -gen-1	155	41	2.4	18.3
α -gen-1	155	836	2.4	17.6	α -gen-1	155	39	2.4	17.6
α -gen-1	151	827	2.5	16.3	α -gen-1	151	35	2.5	16.3
α -gen-1	111	824	2.5	15.9	α -gen-1	111	35	2.5	15.9
$\star$ -gen-1	151	822	2.5	15.6	$\star$ -gen-1	151	30	2.5	15.6

Ted	δ	solved	speed	%ref+	Ted	δ	2E+	speed	%ref+
α -gen-1	511	797	1.2	12.1	α -gen-1	111	33	1.3	12.1
α -gen-1	111	797	1.3	12.1	α -gen-1	511	32	1.2	12.1
$\star$ -gen- Σ	155	789	1.0	11.0	α -gen-1	155	28	1.2	11.0
α -gen- Σ	155	789	1.0	11.0	$\star$ -gen- Σ	155	25	1.0	11.0
α -gen-1	155	789	1.2	11.0	$\star$ -ter-1	511	23	2.4	6.2

Struc	δ	solved	speed	%ref+	Struc	δ	2E+	speed	%ref+
$\star$ -ter-1	115	833	3.9	17.2	$\star$ -sub- Σ	115	32	2.9	17.0
α -ter-1	115	832	2.0	17.0	α -sub- Σ	115	32	1.4	16.9
$\star$ -sub- Σ	115	832	2.9	17.0	α -top- Σ	115	31	1.5	16.0
α -sub- Σ	115	831	1.4	16.9	$\star$ -ter-1	115	29	3.9	17.2
$\star$ -sub-1	115	825	3.6	16.0	$\star$ -top- Σ	115	29	2.9	15.6

5 Conclusions and Future Work

We have implemented several new weight functions for E prover based on term similarity with a conjecture. The experiments suggest that our functions have a potential to improve the autoschedule mode of E as they are reasonably complementary with existing heuristics. In order to use our weight functions with the autoschedule mode of E, we would need to (1) find the best performing parameters of our weight functions, (2) find the best combinations of our weight functions with other weight functions, and (3) find the most complementary combinations and create a scheduling strategy. As a future research, we are planning to use parameter-searching methods such as BliStr [7] to achieve this task.

References

1. Alama, J., et al.: Premise selection for mathematics by corpus analysis and kernel methods. *J. Autom. Reason.* **52**(2), 191–213 (2014)
2. Kaliszyk, C., Urban, J., Vyskocil, J.: Efficient semantic features for automated reasoning over large theories. In: *IJCAI*, vol. 15 (2015)
3. Leskovec, J., Rajaraman, A., Ullman, J.D.: *Mining of Massive Datasets*, 2nd edn. Cambridge University Press, Cambridge (2014)
4. Levenshtein, V.I.: Binary codes capable of correcting deletions, insertions and reversals. *Sov. Phys. Dokl.* **10**, 707 (1966)
5. McCune, W.W.: *Otter 3.0 Reference Manual and Guide*, vol. 9700. Argonne National Laboratory, Argonne (1994)
6. Schulz, S.: E - a brainiac theorem prover. *AI Commun.* **15**(2), 111–126 (2002)
7. Urban, J.: BliStr: the blind strategymaker. In: *Global Conference on Artificial Intelligence, GCAI 2015*, vol. 36, pp. 312–319. EasyChair (2015)
8. Zhang, K., Shasha, D.: Simple fast algorithms for the editing distance between trees and related problems. *SIAM J. Comput.* **18**(6), 1245–1262 (1989)